



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Informe Final

**Propuesta del Proyecto DataQuest:
Validador de Normalización de Bases de Datos Relacionales**

Curso: Base de Datos II

Docente: Ing. Patrick Jose Cuadros Quiroga

Integrantes:

Dongo Palza, Manuel Andre

(2023076802)

Perez Peralta, Fabrizio Salvador Elias

(2023077476)

**Tacna – Perú
2026**

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MADP	Pendiente de revisión docente	Pendiente	21/05/2026	Versión base del informe final DataQuest.

ÍNDICE GENERAL

1. Resumen – Abstract (Inglés).....	4
2. Antecedentes.....	5
a. Contexto académico de bases de datos relacionales	5
b. Dificultades comunes al aprender normalización.....	5
c. Problemas frecuentes en modelos relacionales mal diseñados	5
d. Herramientas existentes y brecha tecnológica identificada	5
3. Planteamiento del Problema	6
a. Problema.....	6
i. Descripción del problema	6
ii. Árbol de problemas.....	6
iii. Causas del problema	6
iv. Efectos del problema	7
b. Justificación.....	7
Justificación técnica	7
Justificación económica.....	7
Justificación social y académica	7
Justificación ambiental	7
Justificación metodológica.....	7
c. Alcance	7
i. Alcance funcional	7
ii. Alcance técnico	8
iii. Fuera del alcance	8
4. Objetivos	8
a. Objetivo general	8
b. Objetivos específicos.....	8
5. Marco Teórico.....	9
a. Bases de datos relacionales	9
b. Modelo relacional	9
c. Claves primarias y claves foráneas	9

d. Dependencias funcionales	9
e. Primera Forma Normal	10
f. Segunda Forma Normal	10
g. Tercera Forma Normal	10
h. Anomalías de inserción, actualización y eliminación	10
i. SQL y sentencias CREATE TABLE.....	10
j. Parsing SQL	10
k. Python aplicado al análisis académico	10
l. Streamlit como interfaz web	10
m. FastAPI como API REST	10
n. Supabase/PostgreSQL como persistencia	10
o. Docker, GitHub Actions y Terraform.....	10
p. QA y pruebas de software	11
q. Seguridad básica en APIs académicas	11
6. Desarrollo de la Solución	11
a. Análisis de Factibilidad	11
b. Tecnología de Desarrollo.....	11
c. Metodología de implementación.....	12
d. Requerimientos funcionales consolidados	12
e. Requerimientos no funcionales consolidados	12
f. Endpoints API principales	12
g. Diagramas y modelos del sistema	13
h. Evidencia visual del sistema	17
i. Casos de uso detallados.....	17
j. Plan de pruebas y QA.....	17
7. Cronograma	18
8. Presupuesto	19
Retorno estimado académico	19
9. Conclusiones.....	19
Recomendaciones.....	20
Bibliografía	20
Anexos	20
Anexo 01 Informe de Factibilidad.....	20
Anexo 02 Documento de Visión.....	20
Anexo 03 Documento SRS	20
Anexo 04 Documento SAD / Arquitectura	21

Anexo 05 Manual de ejecución resumido	21
Anexo 06 Evidencia de pruebas	21
Anexo 07 Checklist QA final	21
Anexo 08 Análisis técnico del repositorio DataQuest.....	22
Anexo 09 Casos de uso detallados	22
CU-01 Ingresar esquema SQL	22
CU-02 Analizar sentencias CREATE TABLE	22
CU-03 Detectar incumplimiento de 1FN.....	23
CU-04 Detectar incumplimiento de 2FN.....	23
CU-05 Detectar incumplimiento de 3FN.....	23
CU-06 Generar sugerencias de corrección	23
CU-07 Exportar reporte	23
CU-08 Consumir API de normalización.....	23
CU-09 Consultar historial	24
CU-10 Usar extensión VS Code	24
Anexo 10 Matriz ampliada de pruebas.....	24
Anexo 11 Ejemplos de consumo API y CLI.....	24
Anexo 12 Matriz de riesgos técnicos	25
Anexo 13 Roadmap de mejora técnica	25
Anexo 14 Manual técnico de instalación y verificación	25

1. Resumen – Abstract (Inglés)

This final report presents DataQuest, an academic and technical platform designed to help students and beginner database designers analyze relational database schemas and identify normalization issues up to Third Normal Form. The project responds to a frequent learning problem in database courses: many students can write SQL tables, but they struggle to determine whether their design contains repeated groups, missing primary keys, partial dependencies or transitive dependencies. Manual review is useful, but it can be slow, subjective and difficult to document during an academic evaluation.

DataQuest integrates a Streamlit web interface, a Python core engine, a FastAPI REST service, a Supabase/PostgreSQL persistence layer, report generation utilities, a command-line interface and a Visual Studio Code extension. The main API endpoint, POST /api/normalizacion, receives SQL CREATE TABLE statements and returns a structured JSON diagnosis with the detected normalization level, violations in 1NF, 2NF and 3NF, optional improvements and a summary. The web interface allows users to paste or load schemas, review results, generate suggestions, download evidence and consult validation history when the Supabase configuration is available.

The solution is not intended to replace a database professor, a complete conceptual modeling process or a formal mathematical proof of all functional dependencies. Its purpose is educational and assistive: it explains likely problems, supports traceable feedback and encourages better relational design practices. The feasibility analysis shows that the project is technically viable because it uses Python 3.10+, Streamlit, FastAPI, PostgreSQL/Supabase, Docker, GitHub Actions and Terraform. From an academic perspective, the project is economically feasible because most technologies are

open source or available through free tiers. The QA review also identifies improvement areas, especially the need to strengthen real automated tests, restrict CORS in production, avoid exposing internal error details and complete formal algorithms for candidate keys and attribute closure.

Keywords: DataQuest, Database Normalization, 1NF, 2NF, 3NF, Relational Databases, Python, Streamlit, FastAPI, PostgreSQL, Supabase, SQL Parser, Academic Software, QA.

2. Antecedentes

a. Contexto académico de bases de datos relacionales

En los cursos de bases de datos, el diseño relacional constituye una competencia central porque permite transformar necesidades de información en estructuras organizadas, consistentes y mantenibles. Sin embargo, la construcción de tablas no se limita a escribir sentencias CREATE TABLE; también exige comprender claves primarias, claves foráneas, atributos atómicos, dependencias funcionales y reglas de normalización. En este contexto, DataQuest surge como una herramienta académica orientada a fortalecer la comprensión práctica de la normalización mediante análisis automático y retroalimentación guiada.

b. Dificultades comunes al aprender normalización

La normalización suele ser difícil para estudiantes porque combina teoría formal con decisiones de diseño. Un alumno puede identificar columnas y tablas, pero no siempre puede explicar por qué una tabla incumple 1FN, por qué una clave compuesta genera una dependencia parcial o por qué un atributo no clave ocasiona dependencia transitiva. Esta dificultad aumenta cuando los ejercicios se revisan manualmente y no existe una evidencia clara del proceso de corrección.

c. Problemas frecuentes en modelos relacionales mal diseñados

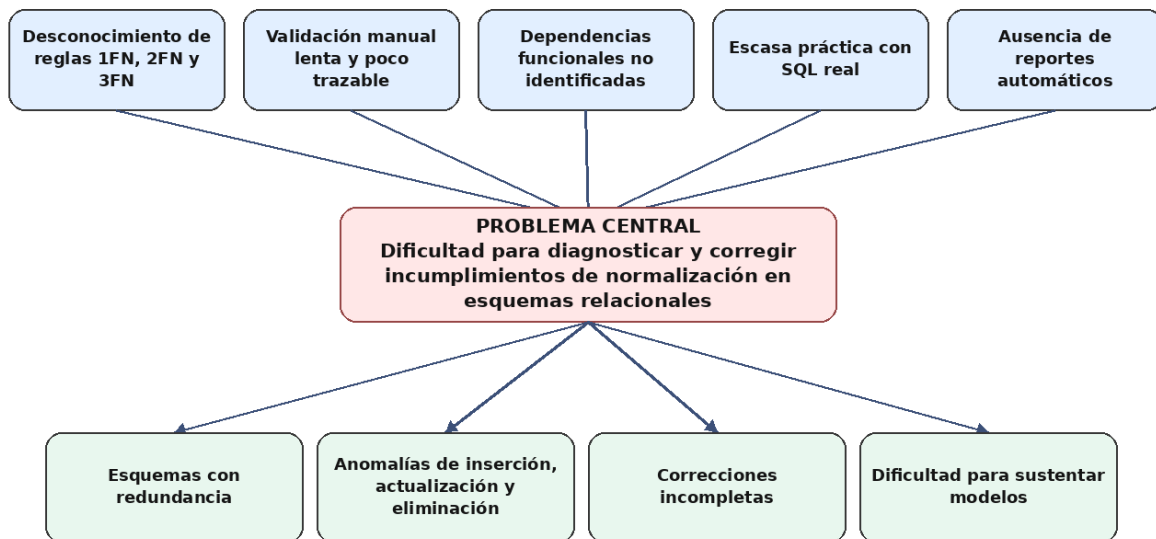
- Redundancia de datos por almacenar información repetida en una misma tabla.
- Atributos multivaluados o grupos repetitivos, como telefonos, emails o columnas numeradas.
- Dependencias parciales cuando una columna depende solo de parte de una clave primaria compuesta.
- Dependencias transitivas cuando un atributo no clave depende de otro atributo no clave.
- Claves primarias ausentes o mal definidas, lo que impide identificar registros de forma única.
- Falta de trazabilidad en la corrección, especialmente cuando no se guardan evidencias del diagnóstico.

d. Herramientas existentes y brecha tecnológica identificada

Existen gestores de bases de datos, modeladores ER y editores SQL que facilitan la creación y visualización de esquemas. No obstante, estas herramientas suelen enfocarse en ejecutar consultas, administrar objetos o dibujar modelos, y no necesariamente en explicar de manera pedagógica por qué un esquema incumple 1FN, 2FN o 3FN. La brecha identificada es la ausencia de una plataforma académica integrada que combine entrada SQL, diagnóstico, sugerencias, API, historial, reportes y uso desde el editor de código.

DataQuest cubre esa brecha desde un enfoque de apoyo al aprendizaje. La herramienta no se presenta como verificador matemático absoluto, sino como un asistente técnico que aplica reglas programadas y heurísticas para orientar la mejora de esquemas relacionales.

Árbol de problemas - DataQuest



Fuente: elaboración propia basada en el análisis del repositorio DataQuest y documentos FD03/FD05.

Figura 1. Árbol de problemas de DataQuest. Fuente: elaboración propia.

3. Planteamiento del Problema

a. Problema

i. Descripción del problema

Los estudiantes y desarrolladores principiantes suelen tener dificultades para identificar si un esquema relacional cumple con Primera Forma Normal, Segunda Forma Normal y Tercera Forma Normal. La normalización requiere interpretar claves, dependencias funcionales, atributos atómicos y relaciones entre tablas. Cuando la revisión se realiza solo de manera manual, el proceso puede ser lento, subjetivo y propenso a errores.

El problema central de DataQuest se define como la dificultad para diagnosticar y corregir incumplimientos de normalización en esquemas relacionales, especialmente cuando el estudiante no cuenta con una herramienta que explique el origen de la violación, sugiera una corrección y conserve evidencia del análisis.

ii. Árbol de problemas

El árbol de problemas identifica como causas principales el desconocimiento de las reglas 1FN, 2FN y 3FN, la falta de herramientas educativas con explicación paso a paso, la validación manual lenta, la dificultad para reconocer dependencias funcionales, la escasa práctica con SQL real y la ausencia de reportes automáticos. Como efectos se presentan esquemas con redundancia, anomalías de inserción, actualización y eliminación, correcciones incompletas, pérdida de tiempo en revisión académica y dificultad para sustentar modelos de base de datos.

iii. Causas del problema

- Desconocimiento de reglas de normalización: el estudiante puede memorizar definiciones, pero no aplicarlas sobre un esquema SQL concreto.
- Validación manual lenta y poco trazable: la revisión tradicional depende de observaciones escritas o explicaciones orales que pueden perderse.

- Dependencias funcionales no identificadas: sin una guía, el alumno no reconoce cuándo un atributo depende de parte de una clave compuesta o de otro atributo no clave.
- Escasa práctica con SQL real: muchos ejercicios se desarrollan en diagramas o tablas teóricas, pero no se validan directamente desde CREATE TABLE.
- Ausencia de reportes automáticos: sin evidencias descargables, es difícil demostrar el estado inicial, los errores encontrados y las mejoras propuestas.

iv. Efectos del problema

- Diseños relacionales con redundancia y menor mantenibilidad.
- Anomalías de inserción, actualización y eliminación en escenarios académicos y prácticos.
- Correcciones incompletas que no atacan la causa de la dependencia.
- Pérdida de tiempo en revisión, sustentación y retroalimentación docente.
- Dificultad para justificar técnicamente por qué un modelo cumple o no cumple 3FN.

b. Justificación

Justificación técnica

DataQuest se justifica técnicamente porque utiliza una arquitectura modular basada en Python, Streamlit, FastAPI, Supabase/PostgreSQL, Docker, GitHub Actions y Terraform. Esta combinación permite separar la interfaz web, el motor de análisis, la API, la persistencia y la automatización de despliegue. El repositorio contiene `app.py` para la navegación Streamlit, `api.py` para el servicio REST, `core/` para el análisis, `db/` para Supabase, `visualizacion/` para diagramas y `vscode-extension/` para integración con Visual Studio Code.

Justificación económica

El proyecto es económicamente viable en un entorno académico porque se apoya principalmente en tecnologías libres o de bajo costo. Python, Streamlit, FastAPI, PostgreSQL, Docker, GitHub Actions y Visual Studio Code pueden utilizarse sin inversión inicial significativa. Supabase puede operar en un plan gratuito para pruebas académicas y Terraform permite documentar infraestructura sin requerir necesariamente despliegue productivo permanente.

Justificación social y académica

La solución contribuye al aprendizaje de bases de datos al entregar retroalimentación inmediata y explicaciones comprensibles. DataQuest permite que estudiantes, docentes y evaluadores revisen un esquema desde varias perspectivas: diagnóstico técnico, sugerencias, reporte descargable, historial y consumo API. Esto mejora la calidad de la práctica académica y reduce la brecha entre teoría de normalización y aplicación real en SQL.

Justificación ambiental

DataQuest favorece una revisión digital de ejercicios, evitando impresiones innecesarias de esquemas, observaciones, correcciones y reportes. El uso de historial digital y reportes descargables reduce el consumo de papel en el entorno académico y facilita el almacenamiento de evidencias.

Justificación metodológica

El sistema permite validar, corregir y documentar de manera trazable. Cada análisis puede vincularse con un esquema original, nivel detectado, violaciones, sugerencias y evidencia. Esta trazabilidad fortalece la evaluación docente y permite comparar el avance del estudiante antes y después de aplicar correcciones.

c. Alcance

i. Alcance funcional

- Ingreso de esquemas mediante SQL pegado o carga de archivos compatibles.
- Análisis de sentencias CREATE TABLE mediante parser en Python.

- Validación de 1FN, 2FN y 3FN mediante reglas programadas y heurísticas.
- Generación de diagnóstico con nivel actual y lista de violaciones.
- Generación de sugerencias de corrección y mejoras opcionales.
- Exportación de resultados en SQL, TXT y PDF desde el módulo de resultados.
- Historial de validaciones en Supabase/PostgreSQL cuando la configuración de entorno está disponible.
- Consumo externo mediante API FastAPI y endpoint POST /api/normalizacion.
- Uso desde Visual Studio Code mediante extensión TypeScript que ejecuta el CLI e inyecta reporte.
- Uso por CLI mediante core/cli.py para agentes, scripts o integración local.

ii. Alcance técnico

- Lenguaje Python 3.10+ para el motor de análisis y Python 3.11 en Dockerfile.
- Interfaz web en Streamlit con navegación por páginas.
- API REST en FastAPI con respuesta JSON.
- Persistencia y autenticación mediante Supabase/PostgreSQL, dependiente de variables de entorno.
- Docker para contenerización de la aplicación web.
- GitHub Actions para pruebas, seguridad y despliegue.
- Terraform para definición de Azure App Service y configuración de despliegue.
- Extensión Visual Studio Code en TypeScript.
- Pruebas unitarias, integración, BDD y UI presentes en estructura, aunque con cobertura limitada.

iii. Fuera del alcance

- No reemplaza a un docente ni una evaluación formal de bases de datos.
- No garantiza interpretación perfecta de todas las dependencias funcionales si el usuario no las declara o si el esquema requiere análisis semántico complejo.
- No sustituye el diseño conceptual completo mediante entrevistas, casos de uso o reglas de negocio.
- No resuelve automáticamente todos los modelos complejos ni demuestra formalmente todas las claves candidatas.
- No debe operar en producción con CORS abierto, credenciales expuestas o manejo de errores con detalles internos.
- No debe afirmar cumplimiento matemático total cuando el motor usa heurísticas.

4. Objetivos

a. Objetivo general

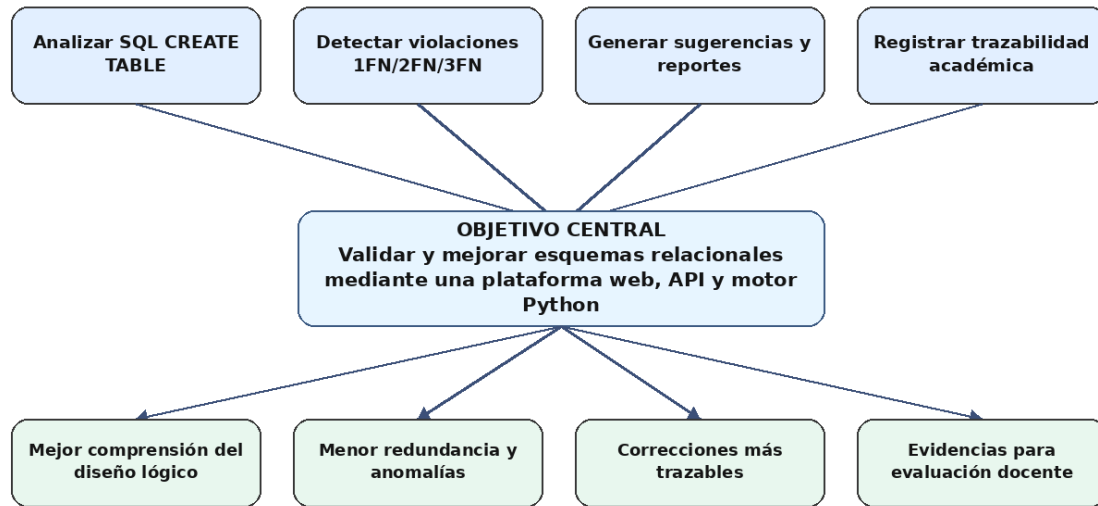
Implementar una plataforma académica denominada DataQuest, orientada a validar y mejorar esquemas de bases de datos relacionales mediante reglas de normalización 1FN, 2FN y 3FN, integrando una interfaz web, un motor lógico en Python, una API FastAPI y persistencia en Supabase/PostgreSQL para generar diagnósticos, sugerencias y evidencias de aprendizaje.

b. Objetivos específicos

- OE1: Analizar esquemas SQL a partir de sentencias CREATE TABLE.
- OE2: Detectar incumplimientos de Primera Forma Normal, Segunda Forma Normal y Tercera Forma Normal.
- OE3: Generar sugerencias de corrección orientadas al aprendizaje.
- OE4: Implementar una interfaz web clara mediante Streamlit.
- OE5: Exponer el motor de normalización mediante API FastAPI.

- OE6: Registrar historial o evidencia de validaciones en Supabase/PostgreSQL cuando el entorno esté configurado.
- OE7: Integrar el uso desde Visual Studio Code y CLI para análisis desde el editor.
- OE8: Validar el sistema mediante pruebas unitarias, pruebas API, checklist QA y casos de entrada correctos e incorrectos.
- OE9: Documentar el proyecto con trazabilidad entre problema, requerimientos, solución y evidencias.

Árbol de objetivos - DataQuest



Fuente: elaboración propia basada en el alcance funcional y técnico de DataQuest.

Figura 2. Árbol de objetivos de DataQuest. Fuente: elaboración propia.

5. Marco Teórico

a. Bases de datos relacionales

Una base de datos relacional organiza la información en tablas compuestas por filas y columnas. Cada tabla representa una entidad o relación del dominio y debe contar con una estructura coherente para evitar duplicidad, inconsistencias y pérdida de integridad. DataQuest toma como entrada estas estructuras mediante sentencias CREATE TABLE y las transforma en un esquema analizable.

b. Modelo relacional

El modelo relacional permite representar datos mediante relaciones, atributos, tuplas y restricciones. En el contexto de DataQuest, el modelo relacional se traduce en tablas, columnas, claves primarias y claves foráneas extraídas por el parser. Esta información es la base para diagnosticar normalización.

c. Claves primarias y claves foráneas

La clave primaria identifica de forma única cada registro de una tabla, mientras que la clave foránea establece vínculos entre tablas. DataQuest detecta claves primarias definidas a nivel de columna o tabla y reconoce claves foráneas básicas mediante REFERENCES o FOREIGN KEY.

d. Dependencias funcionales

Una dependencia funcional $X \rightarrow Y$ expresa que un conjunto de atributos X determina el valor de otro conjunto Y . La normalización depende de estas relaciones. DataQuest permite recibir dependencias funcionales y, cuando no se declaran, usa heurísticas basadas en nombres de columnas para inferir posibles dependencias parciales o transitivas.

e. Primera Forma Normal

La Primera Forma Normal exige que los atributos sean atómicos y que no existan grupos repetitivos. En DataQuest, la validación 1FN identifica tablas sin clave primaria y columnas que sugieren listas, arreglos, grupos repetitivos o sufijos numéricos.

f. Segunda Forma Normal

La Segunda Forma Normal requiere estar en 1FN y evitar dependencias parciales en tablas con clave primaria compuesta. DataQuest revisa si atributos no clave dependen solo de una parte de la clave compuesta, usando dependencias declaradas o inferidas.

g. Tercera Forma Normal

La Tercera Forma Normal exige evitar dependencias transitivas entre atributos no clave. En DataQuest, el validador 3FN identifica dependencias donde un determinante no es superclave y el dependiente no es atributo primo.

h. Anomalías de inserción, actualización y eliminación

Las anomalías aparecen cuando una estructura mal normalizada obliga a repetir datos o impide registrar información sin datos redundantes. DataQuest explica estas situaciones mediante sugerencias asociadas a violaciones de 1FN, 2FN y 3FN.

i. SQL y sentencias CREATE TABLE

SQL permite definir estructuras mediante DDL. DataQuest trabaja principalmente con CREATE TABLE para extraer tablas, columnas, claves y referencias. El parser actual usa expresiones regulares y sqlparse, lo que lo hace práctico para ejercicios académicos, aunque no cubre todos los dialectos SQL avanzados.

j. Parsing SQL

El parsing consiste en transformar texto SQL en una estructura interna. En DataQuest, parse_schema devuelve un diccionario con tablas y dependencias funcionales. Este objeto alimenta el diagnóstico y la generación de reportes.

k. Python aplicado al análisis académico

Python se utiliza como lenguaje principal por su claridad, ecosistema y facilidad para construir herramientas educativas. En DataQuest, Python soporta parser, validadores, corrector, generación de SQL, generación PDF, CLI y API.

l. Streamlit como interfaz web

Streamlit permite construir una interfaz interactiva con formularios, carga de archivos, paneles, resultados y descargas. DataQuest lo usa para ofrecer una experiencia de validación accesible sin requerir conocimientos de frontend complejo.

m. FastAPI como API REST

FastAPI permite exponer el motor DataQuest mediante HTTP/JSON. El endpoint POST /api/normalizacion recibe SQL y responde con nivel actual, violaciones y resumen, facilitando integración con Postman, otros clientes o futuras aplicaciones.

n. Supabase/PostgreSQL como persistencia

Supabase ofrece autenticación, base PostgreSQL y funciones de tiempo real. En DataQuest se utiliza para login, perfiles, presencia e historial de validaciones cuando las variables de entorno están configuradas.

o. Docker, GitHub Actions y Terraform

Docker permite contenerizar la aplicación Streamlit. GitHub Actions define flujos de pruebas, seguridad y despliegue. Terraform describe recursos Azure para App Service, diferenciando aplicación web y API.

p. QA y pruebas de software

La calidad del software requiere pruebas reales, cobertura y revisión de seguridad. DataQuest incluye estructura de pruebas unitarias, integración, UI y BDD, pero el análisis QA identifica que algunas pruebas son placeholders y deben fortalecerse para validar la lógica core.

q. Seguridad básica en APIs académicas

Una API debe controlar CORS, validar entradas, manejar errores sin exponer detalles internos y proteger credenciales. DataQuest incluye validación básica de entrada en api.py, pero se recomienda cerrar CORS en producción y estandarizar errores.

6. Desarrollo de la Solución

a. Análisis de Factibilidad

Tipo de factibilidad	Evaluación	Argumento	Conclusión
Técnica	Alta	El repositorio contiene arquitectura modular con Streamlit, FastAPI, core Python, Supabase, Docker, GitHub Actions, Terraform y extensión VS Code. La solución puede ejecutarse localmente con dependencias correctas.	Viable para entorno académico, con necesidad de robustecer parser y pruebas.
Económica	Viable	Las tecnologías principales son libres o de bajo costo. Supabase, GitHub y herramientas Python permiten prototipo académico sin inversión alta.	El presupuesto referencial es defendible para un proyecto universitario.
Operativa	Media-alta	La interfaz web y la extensión VS Code facilitan el uso por estudiantes. El historial y autenticación dependen de configuración Supabase.	Requiere manual de ejecución, variables de entorno y capturas reales.
Legal	Media	El proyecto usa dependencias open source y debe cuidar licencias, privacidad y credenciales. No maneja datos sensibles reales por defecto.	Viable si se mantiene .env fuera del repositorio y se respetan licencias.
Social/Académica	Alta	Apoya el aprendizaje de normalización y permite evidenciar diagnósticos y correcciones.	Alta utilidad para cursos de Base de Datos y Programación.
Ambiental	Alta	Sustituye revisión impresa por evidencia digital, reportes descargables e historial.	Contribuye a reducir uso de papel en prácticas académicas.

b. Tecnología de Desarrollo

Componente	Tecnología	Uso en DataQuest	Estado	Observación técnica
Python	3.10+ / 3.11 Docker	Motor core, CLI, reportes y backend API	Implementado	Requiere dependencias instaladas para ejecución completa.
Streamlit	1.54.0	Interfaz web, navegación y visualización	Implementado	app.py y views/ organizan páginas.
FastAPI	0.111+	API REST de normalización	Implementado	Endpoint POST /api/normalizacion operativo a nivel de código.
Supabase/PostgreSQL	Supabase 2.5.1	Auth, perfiles, presencia e historial	Parcial	Depende de variables de entorno y tablas db/modelos.sql.
SQL Parser	sqlparse + regex	Extracción de tablas, columnas, PK y FK	Implementado parcial	Parser rudimentario; no cubre todos los dialectos.
Docker	python:3.11-slim	Contenerización de Streamlit	Implementado	Dockerfile expone puerto 8501.
GitHub Actions	tests/security/deploy	Automatización CI/CD	Implementado parcial	Workflows presentes; pruebas reales deben ampliarse.
Terraform	AzureRM	Infraestructura Azure App Service	Planificado/implementado como IaC	Requiere secretos y credenciales Azure.
VS Code Extension	TypeScript	Ejecución CLI e inyección de reporte	Implementado	package.json y extension.ts presentes.
Pytest / Playwright / BDD	requirements-dev	QA automatizado	Parcial	Tests actuales incluyen placeholders/dummy.

c. Metodología de implementación

La implementación se organiza siguiendo la documentación académica FD01-FD05. El FD01 justifica la factibilidad técnica, económica, operativa, social, legal y ambiental. El FD02 define visión, usuarios y alcance. El FD03 especifica requerimientos funcionales, no funcionales, reglas, casos de uso y arquitectura lógica. El FD04 describe decisiones arquitectónicas, componentes y despliegue. Finalmente, el FD05 consolida el informe final con problema, solución, evidencias, QA, cronograma, presupuesto, conclusiones y anexos.

Para DataQuest, la metodología aplicada es incremental: primero se define el motor core de análisis, luego la interfaz Streamlit, posteriormente la API FastAPI, la persistencia, la extensión VS Code y los mecanismos de automatización. El informe final diferencia lo implementado, lo parcialmente implementado y lo recomendado para futuras mejoras.

d. Requerimientos funcionales consolidados

ID	Requerimiento	Módulo	Estado	Prioridad	Evidencia esperada
RF-01	El sistema debe permitir ingresar o cargar esquemas SQL.	Web	Implementado	Alta	Vista Validador con entrada SQL/archivo.
RF-02	El sistema debe parsear sentencias CREATE TABLE.	Core	Implementado parcial	Alta	core/parser.py.
RF-03	El sistema debe validar cumplimiento de 1FN.	Core	Implementado	Alta	core/validador_1fn.py.
RF-04	El sistema debe validar cumplimiento de 2FN.	Core	Implementado	Alta	core/validador_2fn.py.
RF-05	El sistema debe validar cumplimiento de 3FN.	Core	Implementado	Alta	core/validador_3fn.py.
RF-06	El sistema debe generar diagnóstico consolidado.	Core/API	Implementado	Alta	core/diagnostico.py y api.py.
RF-07	El sistema debe generar sugerencias de corrección.	Core	Implementado	Alta	core/corrector.py.
RF-08	El sistema debe generar SQL corregido o refactorizado.	Core/Web	Implementado parcial	Media	core/generador_sql.py.
RF-09	El sistema debe exportar reporte TXT, SQL y PDF.	Web/Core	Implementado	Media	views/05_resultados.py y core/generador_pdf.py.
RF-10	El sistema debe guardar historial de validaciones.	DB/Web	Parcial	Alta	db/modelos.sql y views/06_historial.py.
RF-11	El sistema debe exponer API REST.	API	Implementado	Alta	POST /api/normalizacion.
RF-12	El sistema debe permitir uso por CLI.	Core	Implementado	Media	core/cli.py.
RF-13	El sistema debe integrarse con VS Code.	Extensión	Implementado	Media	vscode-extension/src/extension.ts.
RF-14	El sistema debe mostrar dashboard de métricas.	Web/DB	Parcial	Media	controllers/dashboard_controller.py.
RF-15	El sistema debe incluir pruebas QA.	Tests	Parcial	Alta	Pruebas existen, pero con cobertura insuficiente.

e. Requerimientos no funcionales consolidados

ID	Categoría	Requerimiento no funcional	Prioridad	Criterio de aceptación
RNF-01	Usabilidad	La interfaz debe ser clara para estudiantes principiantes.	Alta	Validación visual de Streamlit y mensajes comprensibles.
RNF-02	Rendimiento	El análisis de esquemas académicos debe responder en pocos segundos.	Media	Pruebas con esquemas de tamaño pequeño y mediano.
RNF-03	Seguridad	La API no debe operar en producción con CORS wildcard.	Alta	CORS restringido por dominios autorizados.
RNF-04	Mantenibilidad	El código debe separar vistas, controladores, core, db y visualización.	Alta	Estructura de carpetas respetada.
RNF-05	Portabilidad	Debe ejecutarse localmente y en contenedor Docker.	Media	Docker build y streamlit run.
RNF-06	Trazabilidad	Cada validación debe conservar esquema, nivel, violaciones y sugerencias.	Alta	Historial en PostgreSQL/Supabase.
RNF-07	Interoperabilidad	La API debe responder JSON estructurado.	Alta	Prueba Postman/curl de POST /api/normalizacion.
RNF-08	Calidad académica	El diagnóstico debe explicar el motivo de cada violación.	Alta	Mensajes de violación y sugerencias.
RNF-09	Pruebas	Las pruebas deben cubrir parser, validadores, API y UI.	Alta	Pytest, Playwright y BDD con casos reales.
RNF-10	Documentación	README, FD03, FD05 y manual deben mantenerse sincronizados.	Alta	Revisión documental antes de entrega.

f. Endpoints API principales

Método	Ruta	Descripción	Entrada	Salida	Estado
GET	/	Verifica disponibilidad de la API.	Sin entrada.	status, message, docs_url	Implementado

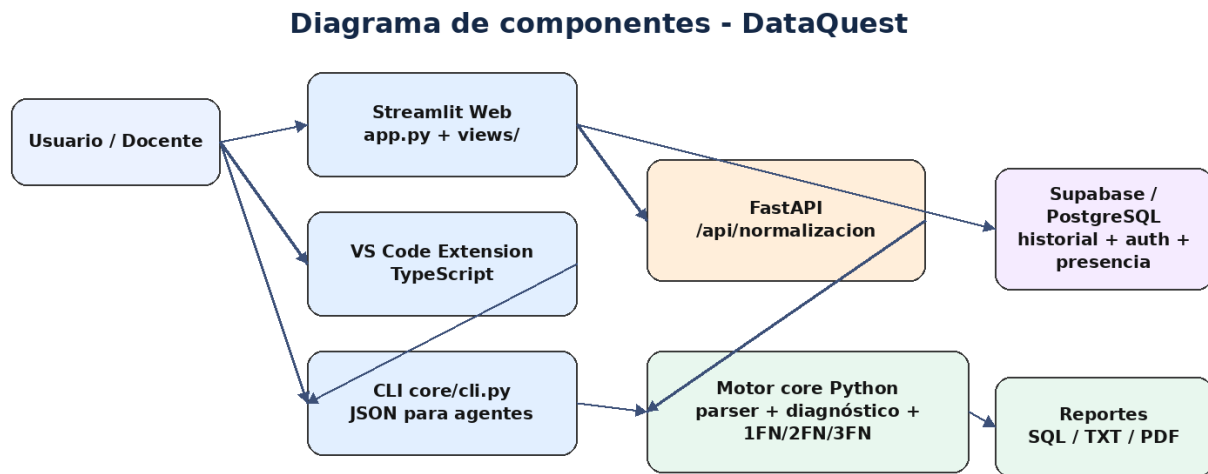
POST	/api/normalizacion	Analiza SQL y devuelve diagnóstico de normalización.	{ "sql": "CREATE TABLE ..." }	success, nivel_actual, violaciones_1fn, violaciones_2fn, violaciones_3fn, mejoras_opcionales, resumen	Implementado
------	--------------------	--	-------------------------------	---	--------------

Ejemplo de prueba sugerida con curl: curl -X POST http://localhost:8000/api/normalizacion -H "Content-Type: application/json" -d '{"sql": "CREATE TABLE alumno (id INT PRIMARY KEY, nombre VARCHAR(50));"}'. La respuesta esperada debe ser JSON y no HTML.

Hallazgos API y seguridad

- api.py incluye CORS con `allow_origins=["*"]`, útil para pruebas académicas, pero inseguro para producción.
- El endpoint devuelve mensajes de excepción dentro del error 500; debe reemplazarse por mensajes controlados sin detalle interno.
- La validación de entrada vacía está implementada y responde `success=False` con código 400 dentro del JSON.
- Se recomienda documentar formalmente la API con OpenAPI/Swagger y ejemplos de petición/respuesta.

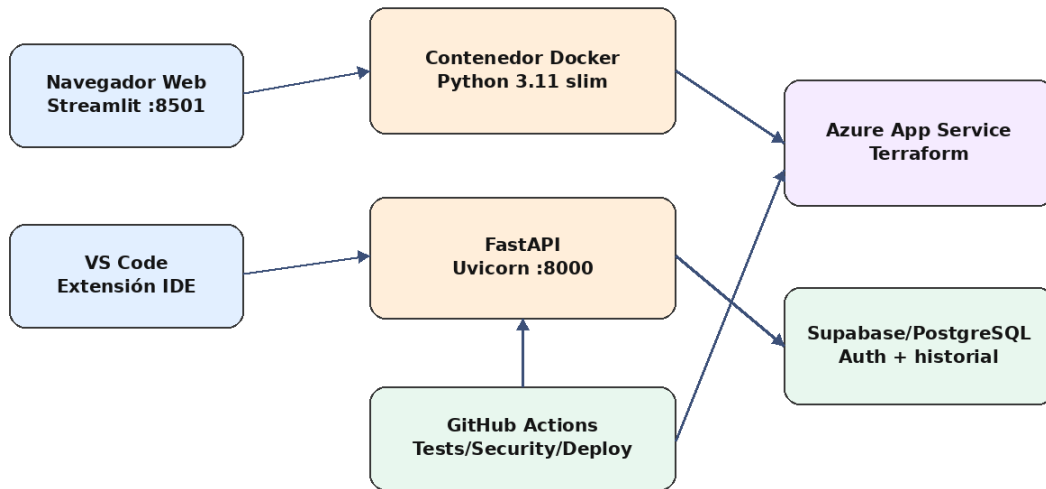
g. Diagramas y modelos del sistema



Fuente: elaboración propia basada en README.md, app.py, api.py, core/, db/ y vscode-extension/.

Figura 3. Diagrama de componentes de DataQuest. Fuente: elaboración propia.

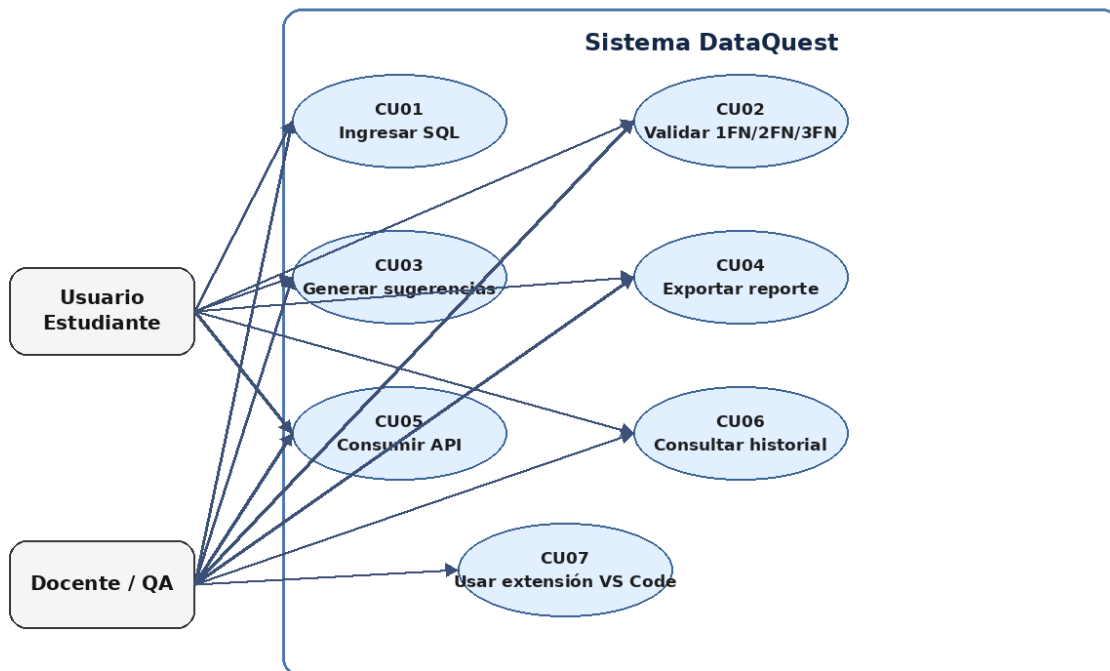
Diagrama de despliegue - DataQuest



Fuente: elaboración propia basada en Dockerfile, workflows GitHub Actions y archivos Terraform del repositorio.

Figura 4. Diagrama de despliegue de DataQuest. Fuente: elaboración propia.

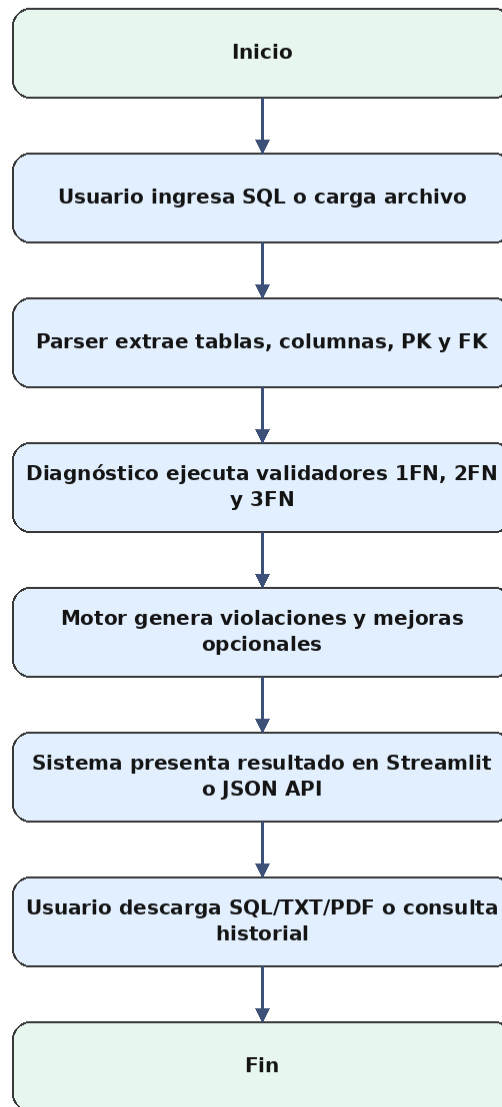
Casos de uso generales - DataQuest



Fuente: elaboración propia basada en funcionalidades de Streamlit, FastAPI, CLI y extensión VS Code.

Figura 5. Casos de uso generales de DataQuest. Fuente: elaboración propia.

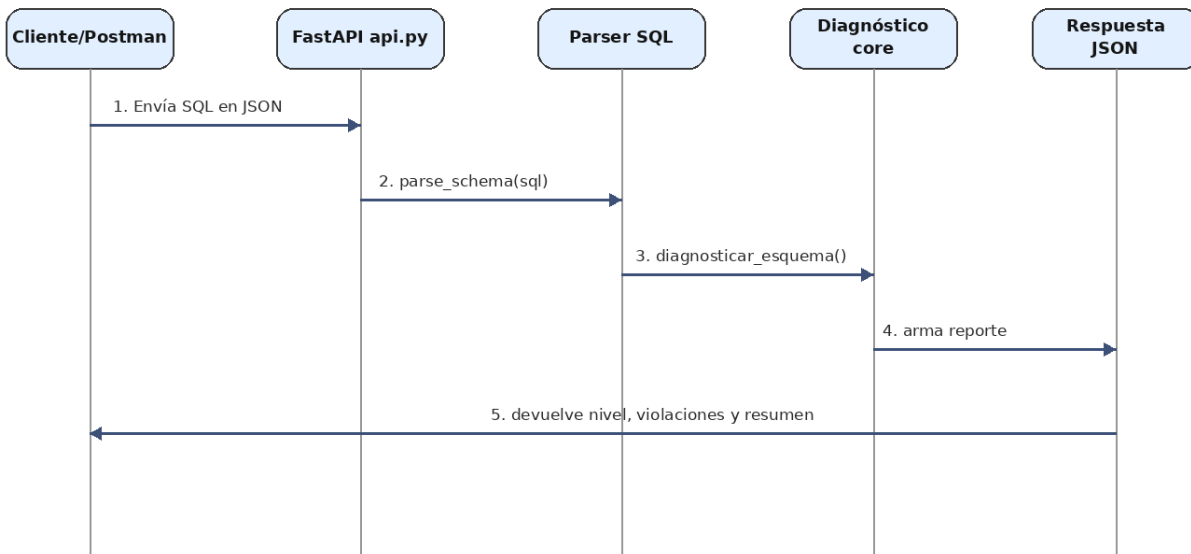
Flujo de validación de normalización



Fuente: elaboración propia basada en core/parser.py, core/diagnostico.py y api.py.

Figura 6. Actividad principal de validación. Fuente: elaboración propia.

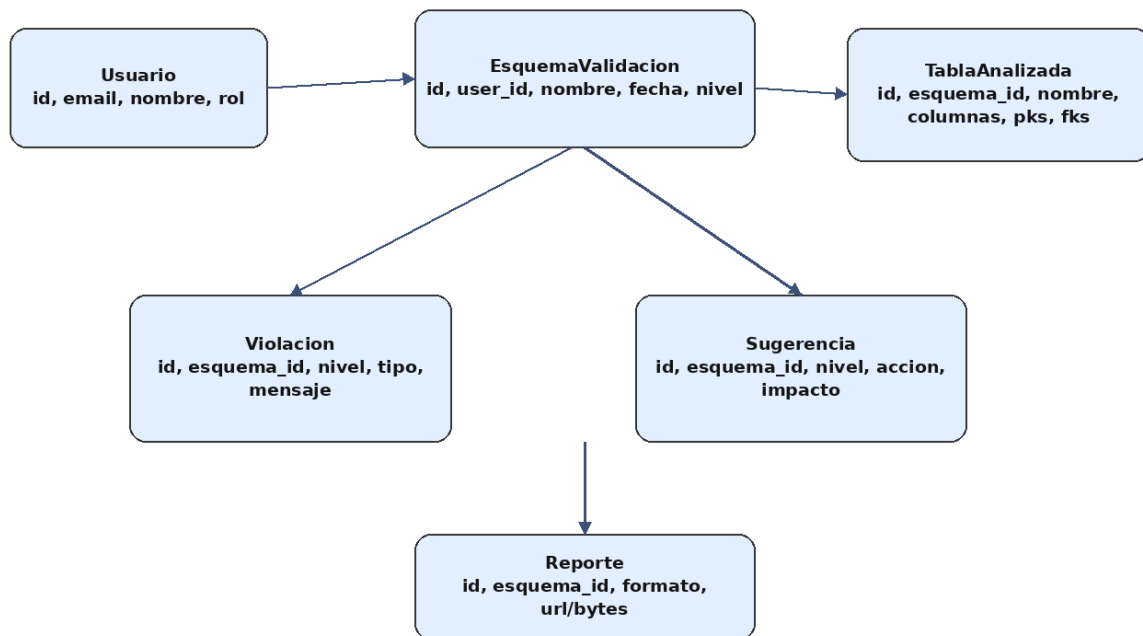
Secuencia de consumo API - POST /api/normalizacion



Fuente: elaboración propia basada en api.py y módulos core del repositorio DataQuest.

Figura 7. Secuencia de consumo de API de normalización. Fuente: elaboración propia.

Modelo lógico conceptual - DataQuest



Fuente: elaboración propia basada en db/modelos.sql, historial_validaciones y entidades del motor DataQuest.

Figura 8. Modelo lógico conceptual de DataQuest. Fuente: elaboración propia.

h. Evidencia visual del sistema

La evidencia visual debe ser obtenida desde el sistema real en ejecución. Para evitar inventar capturas, el presente informe deja identificados los espacios y descripciones que deben completarse con capturas reales durante la sustentación.

Código	Evidencia	Descripción	Estado
EV-01	Pantalla principal Streamlit	Insertar captura de app.py ejecutado con streamlit run app.py.	Pendiente de captura real
EV-02	Módulo Validador	Insertar captura con SQL ingresado y botón de análisis.	Pendiente de captura real
EV-03	Resultado de diagnóstico	Insertar captura mostrando nivel actual y violaciones.	Pendiente de captura real
EV-04	Descarga de reporte PDF/TXT/SQL	Insertar captura de botones de descarga.	Pendiente de captura real
EV-05	FastAPI Docs	Insertar captura de /docs o prueba Postman.	Pendiente de captura real
EV-06	VS Code Extension	Insertar captura de reporte inyectado en archivo SQL.	Pendiente de captura real
EV-07	Historial Supabase	Insertar captura si Supabase está configurado.	Pendiente de captura real
EV-08	GitHub Actions	Insertar captura de ejecución de tests/security/deploy.	Pendiente de captura real

i. Casos de uso detallados

CU	Nombre	Actor	Objetivo	Precondición	Flujo principal	Flujo alternativo	Postcondición	Criterio de aceptación
CU-01	Ingresar esquema SQL	Usuario estudiante	El usuario pega o carga SQL para análisis.	Existe texto SQL válido.	El sistema recibe el esquema y lo envía al parser.	Si está vacío, muestra error controlado.	SQL preparado para diagnóstico.	Debe aceptarse CREATE TABLE.
CU-02	Validar normalización	Usuario estudiante	Ejecutar diagnóstico 1FN/2FN/3FN.	SQL parseado.	El sistema ejecuta diagnosticar_esquema.	Si no hay tablas, informa error.	Reporte generado.	Debe indicar nivel actual.
CU-03	Detectar 1FN	Motor core	Revisar PK y grupos repetitivos.	Tabla detectada.	validador_1fn revisa estructura.	Si no hay PK, registra violación.	Violaciones 1FN listadas.	Debe detectar tabla sin PK.
CU-04	Detectar 2FN	Motor core	Detectar dependencia parcial.	Clave primaria compuesta.	validador_2fn revisa FDs.	Si PK simple, no aplica 2FN parcial.	Violaciones 2FN listadas.	Debe detectar dependencia parcial.
CU-05	Detectar 3FN	Motor core	Detectar dependencia transitiva.	FDs disponibles o inferidas.	validador_3fn analiza determinantes.	Si no hay inferencia, puede requerir FDs.	Violaciones 3FN listadas.	Debe detectar dependencia transitiva.
CU-06	Generar sugerencias	Usuario estudiante	Obtener acciones correctivas.	Diagnóstico generado.	corrector.py crea sugerencias.	Si no hay violaciones, informa sin mejoras críticas.	Sugerencias visibles.	Debe mostrar impacto y beneficios.
CU-07	Exportar reporte	Usuario estudiante	Descargar evidencia.	Resultado calculado.	Sistema genera SQL/TXT/PDF.	Si PDF falla, muestra error.	Archivo descargable.	Debe generar evidencia.
CU-08	Consultar historial	Usuario autenticado	Revisar validaciones anteriores.	Supabase configurado y usuario logueado.	Consulta historial_validaciones.	Si no hay datos, informa vacío.	Historial visible.	Debe filtrar por usuario.
CU-09	Consumir API	Cliente/Postman	Enviar SQL por HTTP/JSON.	API levantada.	POST /api/normalizacion procesa request.	Si JSON inválido, devuelve error.	Respuesta JSON.	Debe responder success true/false.
CU-10	Usar extensión VS Code	Usuario IDE	Analizar archivo desde editor.	Workspace con core/cli.py.	Extensión ejecuta CLI e inyecta reporte.	Si no hay CLI, muestra error.	Reporte en comentario SQL.	Debe no modificar código sin confirmación visible.

j. Plan de pruebas y QA

El análisis QA del repositorio identifica estructura de pruebas, pero también evidencia que varias pruebas son de tipo placeholder. tests/unit/test_core.py y tests/integration/test_db.py contienen pruebas dummy que solo verifican True; tests/ui/test_interface.py y tests/bdd/step_defs/test_login_steps.py incluyen pass. Por ello, el plan QA final debe reforzar la cobertura real del parser, validadores, API, persistencia e interfaz.

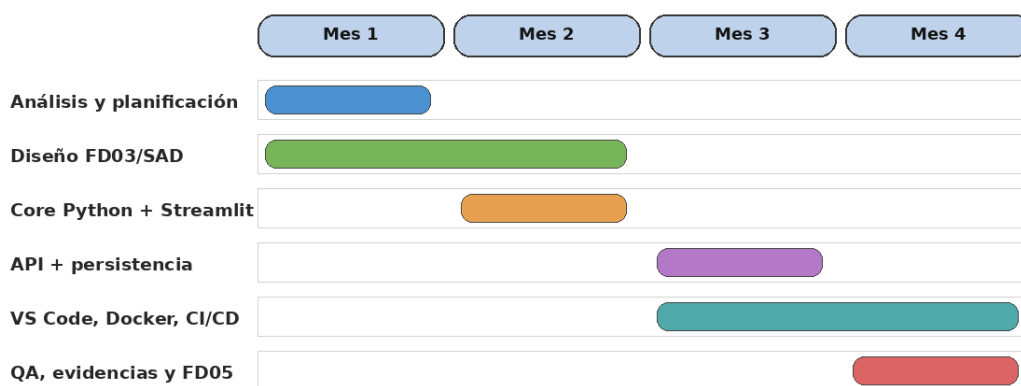
ID	Tipo	Caso	Entrada	Resultado esperado
QA-01	Unitaria	Tabla sin clave primaria	CREATE TABLE alumno(nombre VARCHAR(50));	Debe reportar violación 1FN por ausencia de PK.
QA-02	Unitaria	Atributo multivaluado	Columna telefonos o email1/email2.	Debe reportar grupo repetitivo.
QA-03	Unitaria	Dependencia parcial	PK compuesta con atributo que depende de una parte.	Debe reportar violación 2FN.
QA-04	Unitaria	Dependencia transitiva	departamento_id y departamento_nombre.	Debe reportar violación 3FN.
QA-05	API	SQL válido por POST	JSON con campo sql.	Debe devolver success true y nivel actual.
QA-06	API	SQL vacío	{"sql":""}	Debe devolver success false con código 400.
QA-07	API	JSON inválido	Payload sin campo sql.	Debe devolver error de validación controlado.
QA-08	UI	Streamlit carga	Ejecutar streamlit run app.py.	Debe abrir interfaz sin excepción.
QA-09	DB	Historial por usuario	Supabase configurado.	Debe guardar y listar solo validaciones del usuario.
QA-10	Seguridad	CORS producción	Dominio no autorizado.	Debe bloquear origen no permitido.

Hallazgos QA principales

- Los tests unitarios e integración actuales no validan todavía la lógica de normalización porque contienen asserts dummy.
- El archivo core/dependencias.py contiene funciones TODO para clausura de atributos y claves candidatas, por lo que el análisis formal completo está pendiente.
- El parser es funcional para CREATE TABLE básico, pero debe ampliarse para dialectos SQL complejos.
- La API debe ocultar detalles internos de excepciones y restringir CORS antes de producción.
- La persistencia Supabase debe validarse con variables de entorno reales y Row Level Security correctamente configurado.

7. Cronograma

Cronograma general de 4 meses



Fuente: elaboración propia basada en fases académicas FD01-FD05 y alcance técnico del repositorio.

Figura 9. Diagrama de Gantt general de DataQuest. Fuente: elaboración propia.

Actividad	Responsable	Mes 1	Mes 2	Mes 3	Mes 4	Entregable	Estado
Análisis, alcance y revisión del repositorio	MADP	X				Informe de alcance y hallazgos	Completado
Documentación FD01, FD02 y FD03	MADP	X	X			Documentos base	Completado/ajuste
Desarrollo core parser y validadores	MADP		X	X		Módulos core	Implementado parcial
Interfaz Streamlit y vistas	MADP		X	X		Aplicación web	Implementado
API FastAPI y endpoint principal	MADP			X		api.py	Implementado
Supabase/PostgreSQL e historial	MADP			X		db/modelos.sql y vistas	Parcial
Extensión VS Code y CLI	MADP			X		vscode-extension y core/cli.py	Implementado
Docker, CI/CD y Terraform	MADP			X	X	Dockerfile, workflows, terraform	Parcial
QA, capturas y evidencias	MADP				X	Plan QA y evidencias	Pendiente de completar
FD05 y sustentación	MADP				X	Informe final	En elaboración

8. Presupuesto

El presupuesto es referencial y académico. No constituye una propuesta comercial oficial, pero permite estimar el esfuerzo técnico, documental y operativo necesario para completar DataQuest con calidad de entrega universitaria.

Concepto	Cantidad	Costo unitario	Subtotal
Análisis, revisión de repositorio y documentación base	20 horas	S/ 25.00	S/ 500.00
Desarrollo y ajuste del motor core Python	35 horas	S/ 30.00	S/ 1,050.00
Interfaz Streamlit y experiencia de usuario	30 horas	S/ 30.00	S/ 900.00
API FastAPI, pruebas Postman y documentación	20 horas	S/ 30.00	S/ 600.00
Supabase/PostgreSQL e historial	20 horas	S/ 30.00	S/ 600.00
Extensión VS Code y CLI	25 horas	S/ 30.00	S/ 750.00
Docker, GitHub Actions y Terraform	18 horas	S/ 35.00	S/ 630.00
QA, pruebas, seguridad y correcciones	25 horas	S/ 30.00	S/ 750.00
Diseño de diagramas, evidencias y anexos	18 horas	S/ 25.00	S/ 450.00
Redacción final FD05 y preparación de sustentación	25 horas	S/ 25.00	S/ 625.00
Infraestructura académica y contingencia	1 paquete	S/ 450.00	S/ 450.00
Resumen		Monto	
Costo directo estimado		S/ 6,805.00	
Contingencia académica aproximada 10%		S/ 680.50	
Total referencial del proyecto		S/ 7,485.50	

Retorno estimado académico

El retorno de DataQuest no se mide únicamente en ingresos, sino en valor académico: reducción del tiempo de revisión, mejora en la comprensión de normalización, generación de evidencias y posibilidad de reutilizar el sistema en prácticas futuras. En un escenario institucional, el retorno se expresaría como horas docentes y estudiantiles ahorradas, mayor trazabilidad de entregas y menor repetición de errores comunes de diseño relacional.

9. Conclusiones

- DataQuest responde a una brecha académica real: la dificultad para aplicar normalización de bases de datos sobre esquemas SQL concretos.
- La arquitectura modular permite separar interfaz web, motor core, API, persistencia, reportes y extensión IDE, facilitando mantenimiento y evolución.
- El endpoint POST /api/normalizacion permite integrar el motor con clientes externos, Postman, scripts y futuras aplicaciones.
- La extensión VS Code y el CLI amplían el alcance del sistema hacia el entorno natural de trabajo del estudiante y del desarrollador.
- La solución es técnicamente viable para un entorno académico, pero debe fortalecer pruebas, seguridad API, parser SQL y algoritmos formales de dependencias.
- La documentación FD03 y FD05 mejora la trazabilidad entre problema, requerimientos, implementación, QA y evidencias de sustentación.

Recomendaciones

- Implementar algoritmos formales de clausura de atributos y claves candidatas en core/dependencias.py.
- Permitir ingreso manual de dependencias funcionales desde la interfaz web y API.
- Ampliar pruebas reales para parser, validadores, API, PDF, Supabase y extensión VS Code.
- Restringir CORS en producción y ocultar detalles internos en errores 500.
- Documentar la API con ejemplos OpenAPI/Swagger y colecciones Postman.
- Mejorar la interfaz con mensajes pedagógicos, badges de severidad y cards de diagnóstico.
- Validar Row Level Security en Supabase para que cada usuario solo consulte su historial.
- Mantener sincronizados README, FD03, FD05, manual técnico y evidencias reales.
- Ejecutar pruebas con esquemas correctos, incorrectos, grandes y con dialectos SQL distintos.
- Generar capturas reales antes de la sustentación y colocarlas en la sección de evidencias.

Bibliografía

- Elmasri, R. y Navathe, S. B. Fundamentals of Database Systems. Pearson.
- Silberschatz, A., Korth, H. F. y Sudarshan, S. Database System Concepts. McGraw-Hill.
- Date, C. J. An Introduction to Database Systems. Addison-Wesley.
- Pressman, R. S. Ingeniería del Software: un enfoque práctico. McGraw-Hill.
- Sommerville, I. Software Engineering. Pearson.
- ISO/IEC/IEEE 29148. Systems and software engineering - Life cycle processes - Requirements engineering.

Anexos

Anexo 01 Informe de Factibilidad

La factibilidad técnica, económica, operativa, social, legal y ambiental se resume en la sección 6.a. El resultado general es viable para entorno académico, con recomendaciones de mejora QA y seguridad antes de producción.

Anexo 02 Documento de Visión

La visión del producto es convertirse en una plataforma académica que facilite el aprendizaje, diagnóstico y mejora de esquemas relacionales mediante una experiencia web, API e integración con el entorno de desarrollo.

Anexo 03 Documento SRS

El FD03/SRS de DataQuest especifica requerimientos funcionales, requerimientos no funcionales, reglas de negocio, perfiles, diagramas, API, modelo lógico y criterios de aceptación. Este FD05 consolida esos elementos como informe final.

Matriz de trazabilidad resumida

Elemento	Descripción	Objetivo vinculado	Requerimiento	Evidencia
Problema	Dificultad para diagnosticar normalización	Objetivos OE1-OE3	RF-01 a RF-07	Parser, diagnósticos y sugerencias
Solución Web	Interfaz para validar SQL	OE4	RF-01, RF-06, RF-09	Streamlit app.py y views/
Integración API	Consumo HTTP/JSON	OE5	RF-11	FastAPI /api/normalizacion
Persistencia	Historial y trazabilidad	OE6	RF-10	Supabase/PostgreSQL
IDE/CLI	Uso en entorno de desarrollo	OE7	RF-12, RF-13	core/cli.py y VS Code Extension
Calidad	Pruebas y seguridad	OE8	RNF-03, RNF-09	Plan QA y workflows

Anexo 04 Documento SAD / Arquitectura

La arquitectura propuesta separa clientes, controladores, motor core, persistencia, reportes e infraestructura. Los diagramas de componentes, despliegue, actividad, secuencia y modelo lógico se incluyen en la sección 6.g.

Anexo 05 Manual de ejecución resumido

- Instalar Python 3.10 o superior.
- Crear entorno virtual: `python -m venv .venv`.
- Instalar dependencias: `pip install -r requirements.txt`.
- Configurar `.env` con `SUPABASE_URL`, `SUPABASE_KEY` y `DATABASE_URL` si se usará historial.
- Ejecutar web: `streamlit run app.py`.
- Ejecutar API: `uvicorn api:app --reload --port 8000`.
- Probar CLI: `python core/cli.py archivo.sql`.
- Para VS Code: ingresar a `vscode-extension`, ejecutar `npm install` y abrir modo F5.

Anexo 06 Evidencia de pruebas

Durante la revisión del repositorio se identificó que las pruebas unitarias e integración básicas existen, pero algunas son dummy. Se recomienda reemplazarlas por pruebas reales antes de entrega final. Se deja como evidencia sugerida la ejecución de `pytest`, pruebas API con Postman/curl y capturas del sistema Streamlit.

Anexo 07 Checklist QA final

Elemento QA	Estado	Observación
Parser CREATE TABLE básico	Pendiente de ampliar pruebas	Debe validar tablas simples y compuestas.
Validación 1FN	Implementado	Agregar casos sin PK y grupos repetitivos.
Validación 2FN	Implementado	Agregar casos de clave compuesta.
Validación 3FN	Implementado	Agregar casos con dependencias transitivas.
API FastAPI	Implementado	Restringir CORS y ocultar errores internos.
Supabase historial	Parcial	Validar RLS y variables de entorno.
VS Code Extension	Implementado	Probar instalación y reporte inyectado.
Docker	Implementado	Validar build y run.
GitHub Actions	Parcial	Aumentar cobertura real.
Evidencias visuales	Pendiente	Insertar capturas reales de sustentación.

Anexo 08 Análisis técnico del repositorio DataQuest

Este anexo consolida el análisis técnico del repositorio utilizado para construir el FD05. La finalidad es dejar evidencia de qué partes fueron observadas como implementadas, parciales o pendientes, evitando presentar como terminado aquello que todavía requiere verificación o mejora.

Archivo / carpeta	Tipo	Hallazgo	Estado técnico
README.md	Documentación general	Describe stack, arquitectura MVC, características, uso, extensión VS Code y Agent Skill.	Base documental principal.
app.py	Interfaz web	Configura Streamlit, navegación por páginas y control de sesión.	Implementado.
api.py	API REST	Expone FastAPI, root y POST /api/normalizacion.	Implementado con observaciones de CORS y errores.
core/parser.py	Parser SQL	Extrae tablas, columnas, PK y FK desde CREATE TABLE.	Parser práctico, no completo para todos los dialectos.
core/diagnostico.py	Orquestación del análisis	Ejecuta validadores, infiere dependencias y genera nivel actual.	Implementado con heurísticas.
core/validador_1fn.py	Validación 1FN	Detecta falta de PK y grupos repetitivos por heurística.	Implementado.
core/validador_2fn.py	Validación 2FN	Detecta dependencias parciales en claves compuestas.	Implementado.
core/validador_3fn.py	Validación 3FN	Detecta dependencias transitivas no clave.	Implementado.
core/dependencias.py	Algoritmos formales	Contiene TODO para clausura y claves candidatas.	Pendiente crítico para formalidad avanzada.
core/corrector.py	Sugerencias	Genera sugerencias enriquecidas con impacto, confianza y beneficios.	Implementado.
core/generador_sql.py	SQL propuesto	Genera esquema SQL corregido a partir de sugerencias.	Implementado parcial por heurísticas.
core/generador_pdf.py	Reporte PDF	Genera reporte con resumen, violaciones y sugerencias.	Implementado.
db/modelos.sql	Modelo de datos	Define tablas de perfiles, historial, presencia, RLS y vistas.	Parcial, depende de Supabase real.
vscode-extension/	Extensión IDE	Ejecuta CLI e inyecta reporte como comentario SQL.	Implementado a nivel de código.
tests/	QA automatizado	Incluye unit, integration, UI y BDD.	Parcial; varias pruebas son placeholders.
Dockerfile	Contenerización	Construye imagen Python 3.11 y ejecuta Streamlit.	Implementado.
GitHub Actions	CI/CD	Incluye tests, seguridad y despliegue.	Implementado parcial; requiere secretos.
terraform/	Infraestructura	Define Azure App Service para web y API.	Planificado/implementado como IaC.

Conclusión del análisis técnico

DataQuest tiene una base técnica amplia para un proyecto académico porque integra web, API, CLI, extensión IDE, persistencia, reportes, Docker y automatización. No obstante, el análisis también identifica áreas que deben ser sinceradas en la sustentación: pruebas con baja cobertura, algoritmos formales pendientes, parser SQL limitado y configuración de seguridad API aún en modo desarrollo.

Anexo 09 Casos de uso detallados

CU-01 Ingresar esquema SQL

Campo	Descripción
Actor	Usuario estudiante
Objetivo	Permitir que el usuario proporcione una estructura relacional en formato SQL.
Precondición	El usuario accede al módulo Validador.
Flujo principal	1. El usuario abre el módulo. 2. Pega el SQL o carga un archivo. 3. El sistema conserva el texto para análisis.
Flujo alternativo	Entrada vacía: el sistema debe informar que no existe texto suficiente para validar.
Postcondición	El esquema queda disponible para el parser.
Criterio de aceptación	El sistema debe aceptar al menos una sentencia CREATE TABLE válida.

CU-02 Analizar sentencias CREATE TABLE

Campo	Descripción
Actor	Motor Parser
Objetivo	Extraer información estructural del SQL.
Precondición	Existe texto SQL ingresado.
Flujo principal	1. El parser normaliza espacios. 2. Divide por CREATE TABLE. 3. Extrae nombre, columnas, PK y FK.
Flujo alternativo	Si la sintaxis no contiene tablas, se retorna lista vacía.
Postcondición	Se genera un esquema parseado.
Criterio de aceptación	Debe identificar tablas, columnas y claves primarias básicas.

CU-03 Detectar incumplimiento de 1FN

Campo	Descripción
Actor	Motor core
Objetivo	Evaluar atomicidad y existencia de clave primaria.
Precondición	El esquema contiene al menos una tabla.
Flujo principal	1. El validador revisa PK. 2. Revisa nombres que sugieren listas o repetición. 3. Registra violaciones.
Flujo alternativo	Si no detecta problemas, declara cumplimiento 1FN.
Postcondición	Lista de violaciones 1FN.
Criterio de aceptación	Una tabla sin PK debe ser reportada como violación.

CU-04 Detectar incumplimiento de 2FN

Campo	Descripción
Actor	Motor core
Objetivo	Identificar dependencias parciales en claves compuestas.
Precondición	La tabla tiene clave primaria compuesta.
Flujo principal	1. El sistema analiza FDs declaradas o inferidas. 2. Compara determinantes con subconjuntos de PK. 3. Registra dependencia parcial.
Flujo alternativo	Si la PK es simple, el caso no aplica.
Postcondición	Lista de violaciones 2FN.
Criterio de aceptación	Debe detectar columnas que dependen solo de parte de la clave.

CU-05 Detectar incumplimiento de 3FN

Campo	Descripción
Actor	Motor core
Objetivo	Identificar dependencias transitivas entre atributos no clave.
Precondición	Existen dependencias funcionales o patrones inferibles.
Flujo principal	1. El sistema identifica determinantes no superclave. 2. Verifica dependientes no primos. 3. Registra violación 3FN.
Flujo alternativo	Si la relación no contiene patrón transitivo, no reporta violación.
Postcondición	Lista de violaciones 3FN.
Criterio de aceptación	Debe detectar columna descriptiva dependiente de un ID no clave.

CU-06 Generar sugerencias de corrección

Campo	Descripción
Actor	Usuario estudiante
Objetivo	Entregar recomendaciones aplicables al esquema.
Precondición	Existe un diagnóstico con violaciones.
Flujo principal	1. El corrector lee violaciones. 2. Asigna nivel, acción, impacto y beneficios. 3. Muestra sugerencia.
Flujo alternativo	Si no hay violaciones, solo muestra mejoras opcionales.
Postcondición	Sugerencias disponibles.
Criterio de aceptación	Cada sugerencia debe explicar acción e impacto.

CU-07 Exportar reporte

Campo	Descripción
Actor	Usuario estudiante
Objetivo	Guardar evidencia del análisis.
Precondición	Existe resultado de validación.
Flujo principal	1. El usuario selecciona descarga. 2. El sistema genera TXT, SQL o PDF. 3. El archivo queda disponible.
Flujo alternativo	Si falla PDF, informa error sin detener toda la aplicación.
Postcondición	Reporte descargado.
Criterio de aceptación	Debe incluir nivel, violaciones y sugerencias.

CU-08 Consumir API de normalización

Campo	Descripción
Actor	Cliente externo / Postman
Objetivo	Permitir análisis vía HTTP/JSON.
Precondición	La API se encuentra ejecutándose.
Flujo principal	1. El cliente envía POST. 2. FastAPI valida campo sql. 3. El motor procesa. 4. Responde JSON.
Flujo alternativo	SQL vacío o sin tablas: success false.
Postcondición	Respuesta JSON estructurada.
Criterio de aceptación	Debe responder sin HTML y con campos definidos.

CU-09 Consultar historial

Campo	Descripción
Actor	Usuario autenticado
Objetivo	Revisar validaciones anteriores.
Precondición	Usuario autenticado y Supabase configurado.
Flujo principal	1. El sistema consulta historial_validaciones. 2. Filtra por user_id. 3. Muestra resultados y descargas.
Flujo alternativo	Sin historial: mensaje informativo.
Postcondición	Historial visible.
Criterio de aceptación	No debe mostrar datos de otro usuario.

CU-10 Usar extensión VS Code

Campo	Descripción
Actor	Usuario desarrollador
Objetivo	Analizar un archivo SQL desde el editor.
Precondición	VS Code abierto en workspace con core/cli.py.
Flujo principal	1. El usuario ejecuta comando. 2. La extensión crea archivo temporal. 3. Ejecuta CLI. 4. Inyecta reporte.
Flujo alternativo	Si no hay CLI o JSON inválido, muestra error.
Postcondición	Reporte agregado como comentario SQL.
Criterio de aceptación	Debe preservar el archivo y registrar resultado comprensible.

Anexo 10 Matriz ampliada de pruebas

La siguiente matriz convierte los hallazgos QA en casos verificables. Su objetivo es que la sustentación no dependa únicamente de afirmar que existen pruebas, sino de demostrar qué escenario valida cada caso y qué resultado se considera correcto.

ID	Escenario	Entrada	Resultado esperado	Estado
QA-01	1FN - tabla sin PK	CREATE TABLE producto(nombre VARCHAR(80), precio DECIMAL(10,2));	Detectar violación Sin Primary Key.	Pendiente de automatización real
QA-02	1FN - columnas repetitivas	CREATE TABLE cliente(id INT PRIMARY KEY, telefono1 VARCHAR(20), telefono2 VARCHAR(20));	Detectar grupo repetitivo por sufijo numérico.	Pendiente de automatización real
QA-03	2FN - clave compuesta	CREATE TABLE detalle(id_pedido INT, id_producto INT, producto_nombre VARCHAR(80), cantidad INT, PRIMARY KEY(id_pedido, id_producto));	Detectar producto_nombre como dependencia parcial de id_producto.	Pendiente de automatización real
QA-04	3FN - catálogo embebido	CREATE TABLE empleado(id INT PRIMARY KEY, departamento_id INT, departamento_nombre VARCHAR(80));	Detectar dependencia transitiva departamento_id -> departamento_nombre.	Pendiente de automatización real
QA-05	SQL válido normalizado	CREATE TABLE departamento(id INT PRIMARY KEY, nombre VARCHAR(80));	No reportar violaciones críticas.	Pendiente de automatización real
QA-06	SQL vacío	Cadena vacía	Responder error controlado.	Pendiente de automatización real
QA-07	Sin CREATE TABLE	SELECT * FROM alumno;	No encontrar tablas válidas.	Pendiente de automatización real
QA-08	API JSON válido	POST /api/normalizacion con sql válido	Retornar success true.	Pendiente de automatización real
QA-09	API JSON inválido	POST sin campo sql	Retornar error de validación.	Pendiente de automatización real
QA-10	Historial sin sesión	Abrir historial sin login	Solicitar autenticación.	Pendiente de automatización real
QA-11	CORS producción	Origen externo no permitido	Debe bloquearse cuando CORS se configure correctamente.	Pendiente de automatización real
QA-12	PDF report	Generar PDF con violaciones	Archivo PDF debe incluir resumen y errores.	Pendiente de automatización real
QA-13	VS Code sin workspace	Ejecutar comando sin carpeta abierta	Mostrar error de workspace.	Pendiente de automatización real
QA-14	CLI sin archivo	Ejecutar CLI sin entrada stdin	Retornar error de entrada vacía.	Pendiente de automatización real
QA-15	Docker build	docker build .	Construir imagen sin errores de dependencia.	Pendiente de automatización real

Anexo 11 Ejemplos de consumo API y CLI**Ejemplo de petición API**

```
POST http://localhost:8000/api/normalizacion
Content-Type: application/json
```

```
{
  "sql": "CREATE TABLE alumno (id INT PRIMARY KEY, nombre VARCHAR(80));"
}
```


Ejemplo de respuesta API esperada

```
{
  "success": true,
  "nivel_actual": "3FN",
  "violaciones_1fn": [],
  "violaciones_2fn": [],
  "violaciones_3fn": [],
  "mejoras_opcionales": [],
  "resumen": "El esquema global se encuentra en nivel: 3FN"
}
```

Ejemplo de ejecución CLI

```
python core/cli.py esquema.sql
```

El CLI devuelve JSON estructurado con nivel_actual, violaciones por forma normal, mejoras opcionales y tablas_detectadas. Esta salida se usa como base para la extensión VS Code y para posibles agentes de IA que requieran analizar esquemas automáticamente.

Anexo 12 Matriz de riesgos técnicos

ID	Riesgo	Probabilidad	Impacto	Nivel	Mitigación
R-01	Parser no cubre dialectos SQL complejos	Media	Alta	Alto	Documentar alcance y agregar pruebas por dialecto.
R-02	Pruebas dummy insuficientes	Alta	Alta	Critico	Reemplazar pruebas placeholder por casos reales.
R-03	CORS abierto en API	Alta	Media	Alto	Restringir orígenes antes de producción.
R-04	Errores 500 exponen detalle interno	Media	Media	Medio	Usar manejador global de excepciones.
R-05	Dependencias formales TODO	Alta	Alta	Critico	Implementar clausura y claves candidatas.
R-06	Supabase sin variables de entorno	Media	Media	Medio	Crear manual .env y validación de configuración.
R-07	Credenciales sensibles en despliegue	Baja	Alta	Alto	Usar secrets y nunca subir .env.
R-08	Extensión VS Code depende de ruta local	Media	Media	Medio	Mejorar detección de workspace y Python.
R-09	Reportes generados con caracteres especiales	Media	Media	Medio	Probar UTF-8 y PDF con datos reales.
R-10	Infraestructura Azure no aplicada	Media	Media	Medio	Separar laC planificada de despliegue real verificado.

Anexo 13 Roadmap de mejora técnica

Fase	Horizonte	Acción recomendada	Resultado esperado
Fase 1	Corto plazo	Completar pruebas reales de parser, 1FN, 2FN, 3FN y API.	Cobertura mínima sobre core y api.
Fase 2	Corto plazo	Restringir CORS, estandarizar errores y sanitizar mensajes.	API segura para demostración.
Fase 3	Mediano plazo	Implementar clausura de atributos y claves candidatas.	Diagnóstico más formal.
Fase 4	Mediano plazo	Agregar interfaz para dependencias funcionales manuales.	Mayor precisión en 2FN y 3FN.
Fase 5	Mediano plazo	Completar pruebas de Supabase con RLS.	Historial seguro por usuario.
Fase 6	Largo plazo	Publicar extensión VS Code empaquetada.	Uso instalable por estudiantes.
Fase 7	Largo plazo	Crear panel docente para revisión de entregas.	Valor académico ampliado.
Fase 8	Largo plazo	Soportar dialectos SQL adicionales.	Mayor compatibilidad técnica.

Anexo 14 Manual técnico de instalación y verificación

Instalación local

```
git clone <repositorio-dataquest>
cd proyecto-si783-2026-i-ul-validador-de-normalizacion
python -m venv .venv
.venv\Scripts\activate # Windows
pip install -r requirements.txt
```

Ejecución web

```
streamlit run app.py
```

Ejecución API

```
uvicorn api:app --reload --host 0.0.0.0 --port 8000
```

Pruebas

```
pip install -r requirements-dev.txt
pytest tests/unit tests/integration
pytest tests/bdd
pytest tests/ui
```

Docker

```
docker build -t dataquest .
docker run -p 8501:8501 --env-file .env dataquest
```

Variables de entorno

```
SUPABASE_URL=https://tu-proyecto.supabase.co
SUPABASE_KEY=tu-anon-key
DATABASE_URL=postgresql://usuario:password@host:5432/db
```

Las variables anteriores deben mantenerse fuera del repositorio. El archivo `.env.example` sirve como guía, pero los valores reales deben administrarse como secretos del entorno local, GitHub Actions o Azure.